

Declarative Paradigm: Functional Languages

Declarative programming

- Focus on **what** the result should look like, in terms of its properties
- No implicit state
 - any state must be explicit
- Expressions are evaluated... rather than commands that are executed
- Repetition by recursion not by sequencing commands

Functional Languages

- Emerged in the 1960's with LISP
- Mirrors *mathematical functions*
 - domain = input, range = output
- *Variables* are mathematical *symbols*, not memory locations
- Pure functional programming is *state-free*
- *Referential transparency*: a function's result depends only upon the values of its parameters

Functional Languages

- Computational model is **mathematical function**
 - **Mirrors *mathematical functions***
- The function *Area of Square* has **R** (reals) as domain and range:

- $\text{AreaSquare} : \mathbf{R} \rightarrow \mathbf{R}$

- $\text{AreaSquare}(n) = n^2$

```
areaSquare :: Double -> Double
areaSquare side = side*side
```

```
main = do
    putStrLn (show (areaSquare(5.0)))
```

- In Python

```
areaSquare = lambda side : side*side
```

```
print(areaSquare(5))
```

Functional Languages

- Computational model is **mathematical function**
 - Repetition by **recursion** not by sequencing commands

```
counter :: Int -> Int -> String
counter count maxc
  | count <= maxc = show count ++ "\n" ++ counter (count+1) maxc
  | otherwise = ""
```

defines function **counter**
takes integer arguments **count** and **maxc**
returns string with integers **count** to **maxc** one per line

```
int count = 0;
int maxc = 10;
while (count <= maxc) {
    System.out.println(count) ;
    count = count + 1
}
```

Functional Languages

- Computational model is **mathematical function**
 - Repetition by **recursion** not by sequencing commands

```
counter :: Int -> Int -> String
counter count maxc
  | count <= maxc = show count ++ "\n" ++ counter (count+1) maxc
  | otherwise = ""
```

- Repetition of counter by **recursive** call
- State of counter *explicit* because passed in arguments
- Values of arguments *immutable* within body
- Names like count and maxc (in pure functional language)
 - are **referentially transparent**
 - *always have same value in same context*

Functional Programming

- Everything is **function** (*input->function->output*)
- No **assignments**
 - only constant values, arguments, and returned values
 - No state, memory location
- No loops (only **recursive** functions)
- Referential Transparency:
 - the value of a function depends **only on the values of its parameters.**
 - Evaluation order or execution path don't matter
- Functions are **first-class values**: functions can be parameters and return values

Functional vs. Imperative

	Imperative	Functional
Programmer focus	State changes Step-by-step execution	Information required Transformations to get the result
State changes	Central	Non-existing
Order of execution	Important	Low importance
Primary flow	Loops, method calls, conditions	Function calls (recursion)
Primary manipulation unit	Instances of classes or structures	Functions and data collections

Functional languages

- Pure examples
 - Haskell, Elm, Miranda, Hope, Backus's FP
- Hybrid examples
 - Scala, F#, OCaml, Erlang, Elixir, Lisp, Clojure, Scheme
- Mainstream imperative language with functional subsets
 - Java 8+, C#, Python, Ruby, Groovy, Rust, Swift

**See you in our next
video**